

Computer Programming I

(C++ Programming Language)

Dr. Ahmed Elrefaiy

C++



HOW TO PROGRAM

TENTH EDITION



PAUL DEITEL
HARVEY DEITEL

Introducing
the New C++14
Standard

*Programming is learned
by writing programs.
– Brian Kernighan*

Students' assessment

Examination Method	Marks	Percentage %
Final Written Examination	60	60%
Oral Examination	10	10%
Practical Examination	15	15%
Mid-term Examination	15	15%
Total	100	100%

Agenda

C++ Programming Language

- What is **Programming**
- What is **Language**
- What is a **Programming Language**
- Types of **Languages**
- Types of **Programming**
- **C++** Programming Language
- C++ The **first program**: "Hello, world!"

Programming

- **Program**

- Sequence of instructions written to perform a specific task or solve a problem
- Instructions + Data
- Data like: numbers, text, images, sounds, videos.
- Program Ex. calculators, games, banking software, etc.

- **Programming**

- The Creation process of the program.
- The process of designing, writing, testing, and maintaining code to create a program.

Language

- **Language**

- A system of communication consisting of symbols and rules that allows us to express thoughts, commands, or ideas.

- **In the context of computers**

- It enables humans to interact with machines by instructing what the computer should do.

- **Programming Language**

- Formal language that consists of a set of instructions used to communicate with computers to perform specific tasks and create program

Types of Languages

- Machine language
- Assembly language
- High-Level Language

Machine Language

- The most basic form of language, directly understood by a computer's hardware.
- Written in binary (0s and 1s)
 - 0s and 1s are represented by different **voltage levels** in the hardware of the computer.
 - **0 (Zero) = Low voltage**
 - **1 (One) = High voltage**
 - Ex. 10101101 – 11001010
- **Hard** for humans to read and write

Assembly Language

- Uses English like abbreviations (short, symbolic names) to represent instructions.
- Ex. (to add 10 and 20)
 - Load r, 10 //means: **load** the value **10** into register **r**
 - Add r, 20 //means: **add** the value **20** to the current value in register **r**
 - Store r //means: **store** the value of register **r** into memory
- **A low-level language closer to human language** but still requires knowledge of the computer's architecture
- Requires an **assembler** to translate it into machine code.

High-Level Language

- Ex. (to add 10 and 20)
 - $r = 10 + 20$
- Designed to be **easier for humans** to read, write, and understand.
 - Abstracts away from the hardware specifics of the computer.
- Examples include C, C++, Python, Java, and C#.
- Needs a **compiler** or **interpreter** to convert the code into machine language.

Compiler and Interpreter

- **Compiler**

- A tool that translates a high-level programming language into machine code
- translates the entire program at once and creates an executable file.
- Examples of compiled languages: C++, Java (compiled to bytecode), and C

- **Interpreter**

- A tool that translates high-level code into machine code line-by-line.
- The code is executed directly without creating an executable file.
- Examples of interpreted languages: Python, JavaScript, and Ruby.

Types of Programming Paradigms

- Linear/Sequential Programming
- Structured Programming
- Functional Programming
- Object Oriented Programming (OOP)

Linear/Sequential Programming

- Instructions are executed one after the other in sequence.
- Suitable for simple, straightforward tasks without complex logic.
- Ex. A script that adds two numbers

Functional/Structure Programming

- **Functional**

- *Programs are constructed by applying and composing **functions**.*
- *There is a single-entry point function (often called **main** in many languages).*
- *Ex. Haskell, Lisp, and Scala.*

- **Structure**

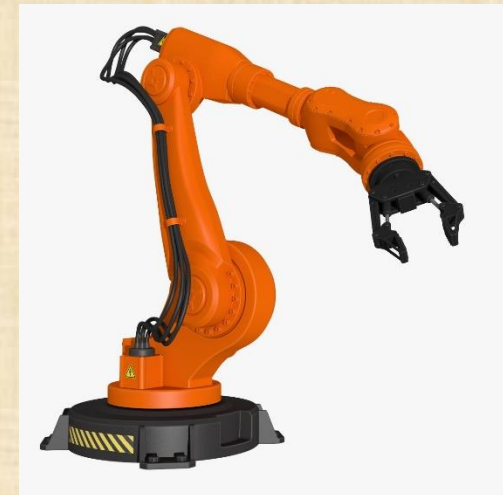
- *improve modularity and organization*
- *focuses on using clear control structures, such as sequential code, loops, and conditionals*
- *Ex. C, Pascal, and early versions of C++.*

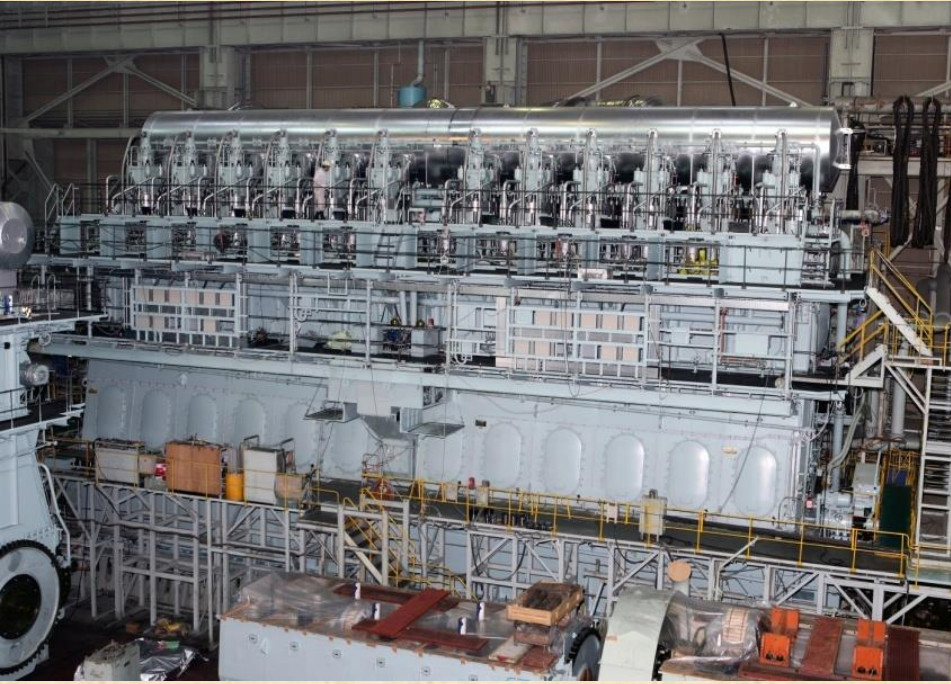
Object-Oriented Programming (OOP)

- Uses objects (instances of classes) to encapsulate data and behavior.
- Promotes concepts like encapsulation, inheritance, and polymorphism.
- Ex. C++, Java, C#, and Python.

Why programming?

- Our civilization runs on software
 - Most engineering activities involve software
- Note: most programs do not run on things that look like a PC
 - a screen, a keyboard, a box under the table





Ships



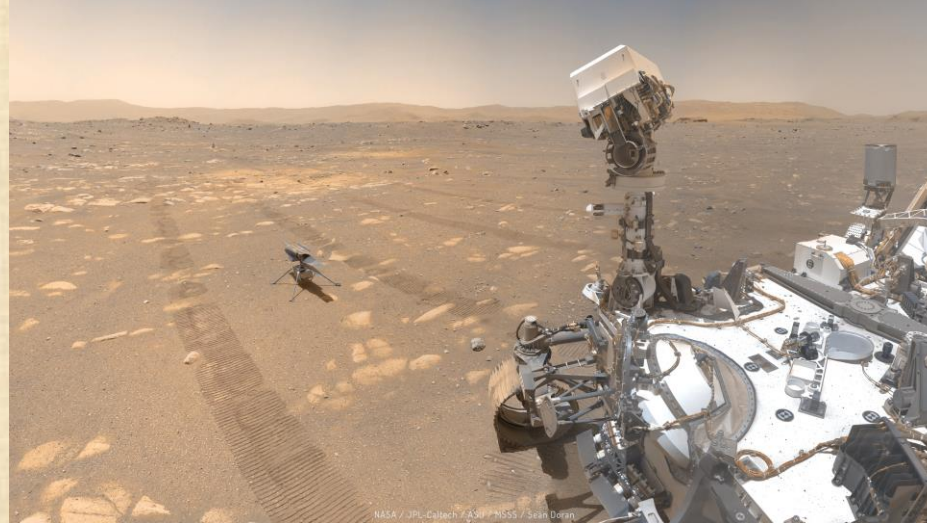
- Design
- Construction
- Management
- Loading
- Scheduling
- Route planning

- Monitoring
- Engine
- Hull design
- Pumps

Aerospace



- Communication
- Control
- Display
- Routing



- Signal processing
- “Gadget” control
- telemetry





Cars

(distributed computers with wheels)

- Gas, diesel, hybrid, electric
- Self-driving
- Navigation
- Entertainment
- Design
- Monitoring
- Steering
- Breaks



Phones

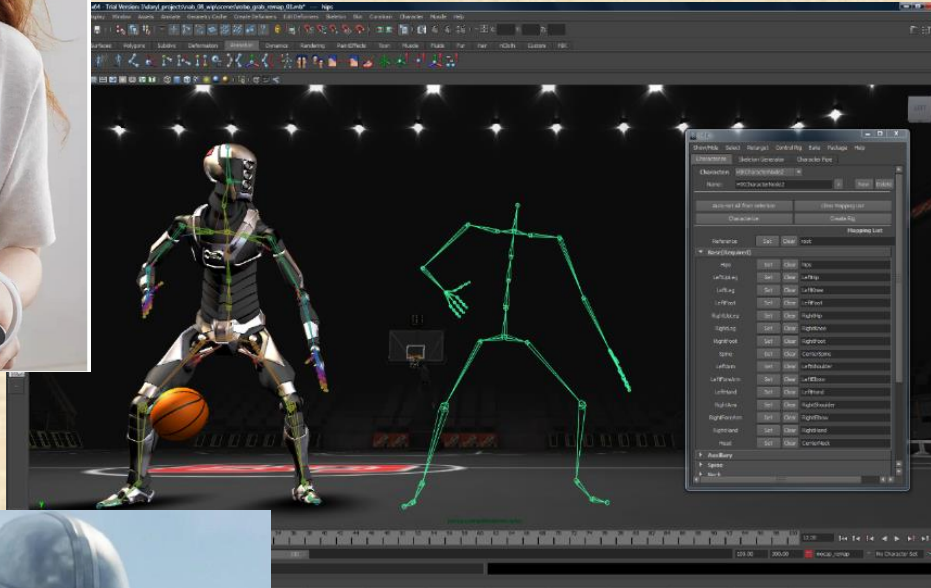


- Voice quality
- User interfaces
- Billing
- Mobility



- Switching
- Reliability
- Provisioning
- Images
- Videos
- apps

Graphics, animation, and games



Commerce, finance, and governance

- Banks
- Stock exchanges
- Currency exchange
- Social services
- Medical records
- Taxes
- Online stores



Medicine



- Scanners
- Vaccine development and production
- Analysis (blood, tissue)
- General research

- Genomics
- Materials design
- Simulations
- ???

Energy

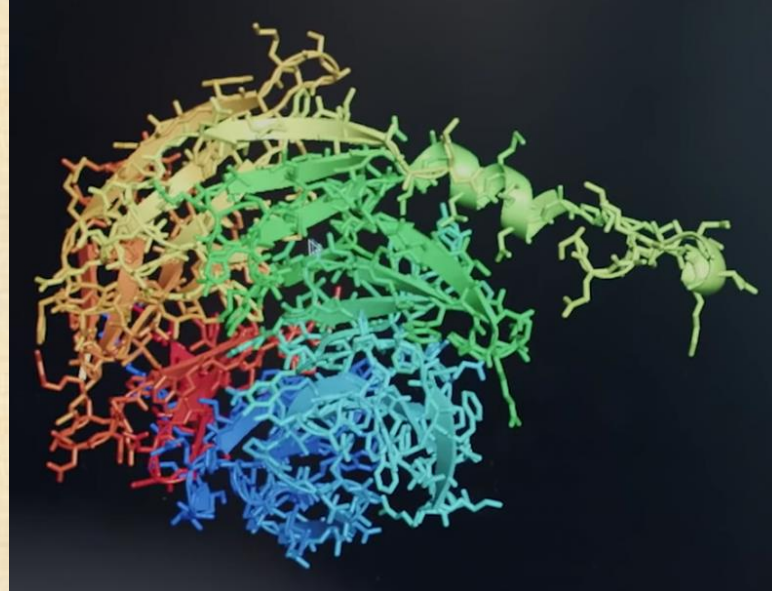


- Control
- Monitoring
- Analysis
- Design

- Communications
- Visualization
- Manufacturing
- Transport

Science

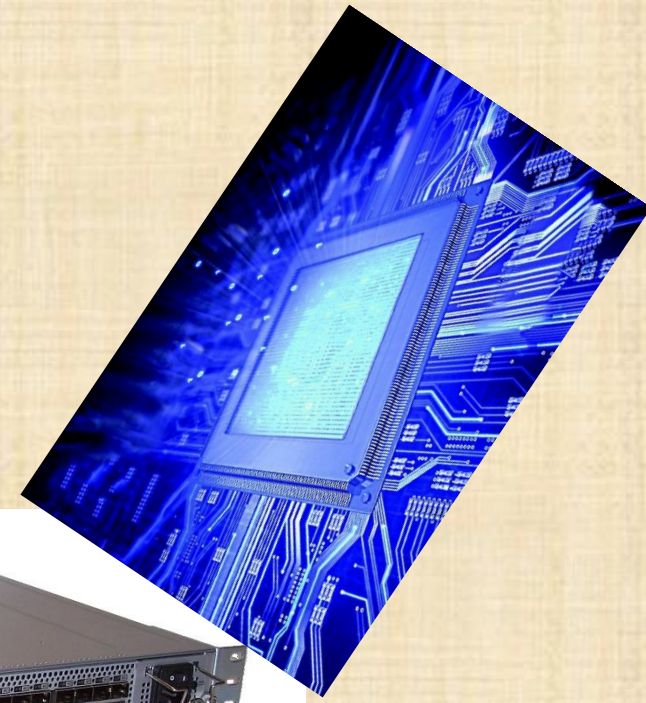
- Physics
- Biology
- Engineering
- Astronomy



Foundations



- Microelectronics
- Computers
- Routers
- Networking
- Manufacturing
- Wireless



- Operating systems
- Browsers
- Virtual machines
- Search engines

Laptops, tablets, workstations, servers, ...



- There's a lot more to computing than games, word processing, browsing, and spreadsheets!

Why C++ ?

- You can't learn to program without a programming language
- The purpose of a programming language is to allow you to express your ideas in code
- C++ is the language that most directly allows you to express ideas from the largest number of application areas
- C++ is the most widely used language in engineering areas
 - <http://www.stroustrup.com/applications.html>
- Other languages often use C++ for computation intensive tasks
 - E.g., Python doing AI
- The implementation of many languages are C++ programs
 - E.g., Java and JavaScript



Why C++ ?

- C++ is precisely and comprehensively defined by an ISO standard
 - And that standard is almost universally accepted
 - The most recent standard is ISO C++ 2023
- C++ is available on almost all kinds of computers
- Programming concepts that you learn using C++ can be used fairly directly in other languages
 - Including C, Java, C#, and (less directly) Fortran

A first program

// a first program:

```
#include <iostream>           // get the library facilities needed for now

int main()                   // main() is where a C++ program starts
{
    std::cout << "Hello, world!\n";    // output the 13 characters Hello, world!
                                       // followed by a new line

    return 0;                 // return a value indicating success
}

// the std:: says that cout comes from the standard library
// main() 's result indicate success (value 0) or failure (non zero value)
```

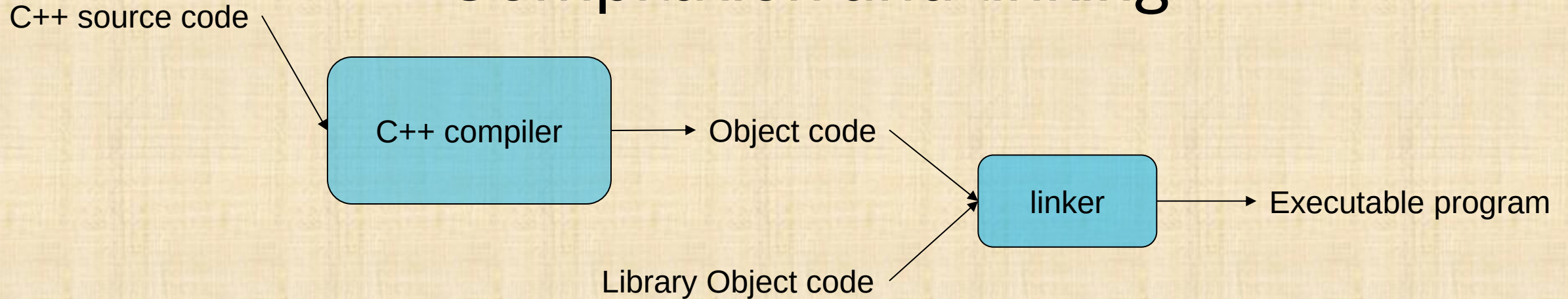
A first program

// a first program:

```
#include <iostream>           // get the library facilities needed for now
using namespace std;
int main()                   // main() is where a C++ program starts
{
    cout << "Hello, world!\n"; // output the 13 characters Hello, world!
                                // followed by a new line
    return 0;                // return a value indicating success
}

// the std:: says that cout comes from the standard library
// main() 's result indicate success (value 0) or failure (non zero value)
```

Compilation and linking



- You write C++ source code (e.g. **Hello.cpp**)
 - Source code is (in principle) human readable
- The compiler translates what you wrote into object code (e.g., **Hello.o**)
 - sometimes called machine code
 - Object code is simple enough for a computer to “understand”
- The linker links your code to other code needed for it to execute
 - E.g., input/output libraries, operating system code, and windowing code
- The result is an executable program
 - E.g., a **.exe** file on windows or an **a.out** file on Linux

The next lecture

- Will talk about types, values, variables, declarations, simple input and output, very simple computations, and type safety.